第 03 章

Embedding、Milvus 与 RAG 检索

把文字变成检索候选，再把候选变成有归属、页码和版本的引用证据。

这一章完成后，你能够

- 区分 Embedding 模型、向量索引、检索服务与生成模型。
- 读懂资料 Collection 的结构、向量维度和 COSINE 排序。
- 追踪 Milvus 候选如何经过 PostgreSQL 归属与有效性复核。
- 运行离线检索与证据实验，明确双索引和混合检索的区别。

学习前提

第 2 章的 DocumentChunk、material_id 与 content_hash；Python 列表、类方法与 async 基础；项目锁定依赖的 apps/api/.venv 环境

代码基准：72dea46 + 当前已完成的学习准备。Python 3.12；LangChain 1.4.3；LangGraph 1.2.12。

正文采用场景、名词、步骤、源码、实验和练习的教学顺序。截图注明数据来源；教学简化与生产实现分别标记。

本章目录

3.1 业务问题：怎样从资料里找到回答当前问题的片段？ 3

3.2 Embeddings 接口：一条查询与一批文档为什么分开？ 4

3.3 余弦相似度：分数表示接近程度，不是答案正确率 6

3.4 Milvus Collection：除了向量，还必须存哪些字段？ 7

3.5 upsert 与线程适配：片段怎样成为索引记录？ 8

3.6 search 与 top_k：范围先固定，再取相似候选 10

3.7 从候选到证据：归属校验与失效复核为什么做两次？ 11

3.8 完整离线实验：检索过滤与证据组装分别验证 14

3.9 双索引与后续优化：先准确描述已有能力 16

3.10 综合练习与参考思路 18

3.11 源码阅读清单与下一步 18

3.1 业务问题：怎样从资料里找到回答当前问题的片段?

用户在学习辅导中问：“这里的连接过程应该怎样理解？”资料可能有几十页，系统不能每次都把整份文件塞给模型。本章解决的是先找到与问题相关的片段，并确认片段仍属于当前知识库、仍然有效。生成模型怎样调用工具和组织回答，留到 Agent 章节展开。



图 3-1 真实项目资料辅导页面。观察知识库范围、用户问题与来源展示；不要把截图中的既有材料替换理解成代码实验的 TCP 短文。

当前真实项目页面 / 现有业务数据 / 2026-10-07。既有辅导回复的原文依据展开；未发送新问题



图 3-2 辅导材料检索的主路径。检索先产生候选，只有通过校验才成为可引用证据。

RAG 是 Retrieval-Augmented Generation，即检索增强生成。它包含检索与生成两个职责：检索负责找到外部知识，生成负责根据问题和知识组织答案。Embedding 只把文字转为数值，不负责写答案；Milvus 只返回相似数据，也不会自行判断用户答题错在哪里。先把职责分清，阅读调用链就容易很多。

组件 / 名词	输入 → 输出	在本项目中的作用
Embedding / 向量化	字符串 → 浮点数列表	给资料和查询建立可以比较的表示
Dense retrieval / 稠密检索	查询向量 → 相似候选	按向量相似度寻找资料块
MilvusDocumentIndex	资料块或查询 → ID / SearchHit	封装资料向量写入与查询
LearningMaterialRetrieval	知识库、问题 → 证据字典列表	回查 PostgreSQL，确认归属和有效内容
生成模型	问题 + 工具结果 → 回答草稿	由辅导 Agent 调用，随后还要校验引用

表 3-1 从转换、查找、校验到表达。类名相似，也要继续比较各自返回类型。

3.2 Embeddings 接口：一条查询与一批文档为什么分开?

LangChain 的 `Embeddings` 接口有两个主要方法。`embed_documents(texts)` 接收多条字符串，返回多条向量；`embed_query(text)` 接收一条查询，返回一条向量。它们必须工作在兼容的向量空间中：写入文档所用的模型、维度和处理方式，应与检索时保持一致。相同维度只是必要条件，不足以证明两个不同模型能够混用。

实现 / 配置	实际行为	适合怎样理解
local	DeterministicEmbeddings，默认 384 维	稳定、离线的开发回退，不能据此衡量生产检索质量
openai_compatible	OpenAIEmbeddings 调兼容端点	接口适配方式，不表示所有端点参数都完全相同

实现 / 配置	实际行为	适合怎样理解
tei	POST /embed，inputs 为文本列表	适配 Text Embeddings Inference 服务
embedding_dimensions	本项目默认 384，可配置	要与模型返回结果、Collection 结构一起核对
embedding_model	写入记录中的模型标签	记录来源；改变模型需要重新评估旧向量

表 3-2 提供者选择以配置和当前源码为准，不把开发回退写成真实语义模型。

源码节选：创建 Embedding 适配器

定位：apps/api/study_agent/ingestion/embeddings.py，第 70-98 行。

```python
def build_embeddings(settings: Settings) -> tuple[Embeddings, str]:
    if settings.embedding_provider == "tei":
        if not settings.embedding_base_url:
            raise ValueError("使用 TEI 时必须配置 EMBEDDING_BASE_URL")
        return (
            TeiEmbeddings(settings.embedding_base_url, settings.embedding_timeout_seconds),
            settings.embedding_model or "tei",
        )
    if settings.embedding_provider == "openai_compatible":
        if not settings.embedding_base_url or not settings.embedding_model:
            raise ValueError(
                "使用 OpenAI-compatible Embedding 时必须配置 EMBEDDING_BASE_URL 和 EMBEDDING_MODEL"
            )
        return (
            OpenAIEmbeddings(
                base_url=settings.embedding_base_url,
                api_key=SecretStr(settings.embedding_api_key or "not-required"),
                model=settings.embedding_model,
                dimensions=settings.embedding_dimensions,
                timeout=settings.embedding_timeout_seconds,
                # 百炼的 OpenAI-compatible 工作空间端点要求 input 为字符串数组，
                # 不接受 LangChain 长文本安全模式生成的 token ID 数组。
                check_embedding_ctx_length=False,
                # text-embedding-v4 当前单次最多接收 10 条文本。
                chunk_size=10,
            ),
            settings.embedding_model,
        )
    return DeterministicEmbeddings(settings.embedding_dimensions), "deterministic-local-v1"
```

settings 是 Settings 对象；返回值为 (Embeddings 实例, 模型名称)。

TEI 和兼容端点都需要必要地址配置；本地回退不产生网络请求。

兼容端点关闭 token ID 输入转换并设批量大小 10；这里的 chunk_size 是 API 批量条数。

这里出现一个容易混淆的参数：`OpenAIEmbeddings(chunk_size=10)` 指每批处理的文本条数；上一章 `RecursiveCharacterTextSplitter(chunk_size=800)` 指字符块大小。同名参数出现在不同类中，单位并不相同。阅读代码时，应先确定对象类型，再理解参数，不能只记住名字。

本地 DeterministicEmbeddings 会把字符、英文词串和连续三字符特征映射到固定桶，并把向量归一化。它可用于稳定地验证接口、过滤和数据流，却没有学习到真实语言语义。生产供应商测试是否通过与检索质量是否足够，也是两个需要分别验证的问题。

真实数据与外部调用

本章实验固定使用内存里的教学文本与本地向量实现，不发送飞书文档和真实学习记录。实际供应商的模型名称、端点与费用应以你使用的配置为准。

3.3 余弦相似度：分数表示接近程度，不是答案正确率

向量可以想成从原点出发的箭头。余弦相似度 COSINE 比较两支箭头的方向：方向越接近，相似度越高。计算方式是点积除以两个向量长度的乘积。两个完全同向的非零向量得到 1，互相垂直得到 0，反向得到 -1。高维向量也是同样的数学关系。



图 3-3 概念示意：文档向量预先建立，查询向量在线生成；二者一起参与比较，不是文档先生成用户问题。

例如二维向量 [1,0] 与 [2,0] 同向，余弦为 1，尽管长度不同。这个小例子说明 COSINE 主要关注方向。真实文本向量每一维通常没有单独可读的“某个知识点”含义，不能看到一个维度变大就解释为模型更懂 TCP。向量的价值来自整体空间关系和实际检索表现。

distance 字段容易读反

pymilvus 返回字段名叫 distance，项目把它转换为 SearchHit.score。当前索引采用 COSINE，值越大表示越相似；不能套用“距离越小越好”的 L2 规则。官方指标说明：https://milvus.io/docs/metric.md

分数为 0.8，不表示“答案有 80% 概率正确”。它只描述这次向量比较；不同模型、语料与度量之间也不能直接横向比较数值。当前资料查询没有设置最低相似度阈值，所以在没有真正相关材料时，也可能返回最接近但不适合回答的候选。Agent 仍应判断是否证据不足。

3.4 Milvus Collection：除了向量，还必须存哪些字段?

Milvus 用 Collection 组织一组结构一致的数据。Field 是字段，Entity 是一条记录，Schema 定义字段类型与约束。项目没有通过 langchain-milvus 的 VectorStore 自动建库，而是在 `MilvusDocumentIndex` 内直接使用 pymilvus 客户端，显式定义资料索引结构。

源码节选：定义资料向量集合与索引

定位：apps/api/study_agent/ingestion/vector_index.py，第 25-53 行。

```python
    def _ensure_collection(self, dimensions: int) -> None:
        if self.client.has_collection(self.collection_name):
            description = self.client.describe_collection(self.collection_name)
            fields = description.get("fields", [])
            vector_field = next((field for field in fields if field.get("name") == "vector"), None)
            existing = (vector_field or {}).get("params", {}).get("dim")
            if existing is not None and int(existing) != dimensions:
                raise ValueError(
                    f"Milvus Collection {self.collection_name} 的向量维度为 {existing}, "
                    f"当前 Embedding 返回 {dimensions} 维；请更换 Collection 名称或重建索引"
                )
            return
        schema = self.client.create_schema(auto_id=False, enable_dynamic_field=False)
        schema.add_field("id", DataType.VARCHAR, max_length=36, is_primary=True)
        schema.add_field("vector", DataType.FLOAT_VECTOR, dim=dimensions)
        schema.add_field("knowledge_base_id", DataType.VARCHAR, max_length=36)
        schema.add_field("material_id", DataType.VARCHAR, max_length=36)
        schema.add_field("content", DataType.VARCHAR, max_length=65535)
        schema.add_field("page_start", DataType.INT64, nullable=True)
        schema.add_field("page_end", DataType.INT64, nullable=True)
        schema.add_field("heading_path", DataType.JSON)
        index_params = self.client.prepare_index_params()
        index_params.add_index(field_name="vector", index_type="AUTOINDEX", metric_type="COSINE")
        self.client.create_collection(
            collection_name=self.collection_name,
            schema=schema,
            index_params=index_params,
            consistency_level="Strong",
        )
```

dimensions 由本次返回向量的长度确定；已有集合会检查 vector 字段维度。

auto_id=False 表示主键由项目提供，id 是 UUID 字符串；dynamic field 关闭。

向量字段使用 AUTOINDEX 和 COSINE；Strong 是 Milvus 的一致性设置。

Field / 配置	类型与约束	为什么需要它
id	VARCHAR，主键，最长 36	对应 DocumentChunk.id，回查业务数据库
vector	FLOAT_VECTOR，dim 固定	保存稠密向量，用于相似度检索

Field / 配置	类型与约束	为什么需要它
knowledge_base_id / material_id	VARCHAR，最长 36	限制范围并核对材料归属
content	VARCHAR，max_length=65535	保存索引侧正文候选
page_start / page_end	可空 INT64	保留引用页码；无页码格式允许为空
heading_path	JSON	保留章节层级信息

表 3-3 资料集合的主要字段。Schema 固定，意味着新增 metadata 不会自动成为可查询字段。

`AUTOINDEX` 是自动选择索引的配置入口，不能据此宣称项目显式采用 HNSW 并调优了参数。`Strong` 约束 Milvus 的查询可见性，也不代表 PostgreSQL 与 Milvus 的写入组成同一个事务。课程展示了更丰富的稀疏向量、BM25 和重排功能，但当前这张 Schema 只有一个稠密向量字段。

已有集合维度为 384，而新模型返回 1024 维时，代码会抛出明确异常，提示换集合名或重建索引。即便新旧模型都返回 384 维，旧向量仍可能不再兼容，因此模型更换还需要处理数据迁移和全量重新向量化。代码中的维度检查不能代替模型一致性管理。

3.5 upsert 与线程适配：片段怎样成为索引记录？

资料解析后，`index_material` 从 PostgreSQL 按序读取 DocumentChunk，调用资料索引的 `upsert`。upsert 可以理解为“主键存在则更新，不存在则插入”。由于片段主键稳定，重复处理可以再次写入相同身份，而不是每次生成一批无关记录。写入成功后，业务记录再保存 vector_id 与 embedding_model。

源码节选：批量向量化并写入 Milvus

定位：apps/api/study_agent/ingestion/vector_index.py，第 55-78 行。

```
55      async def upsert(self, *, knowledge_base_id: UUID, chunks: list[DocumentChunk]) -> list[str]:
56          vectors = await anyio.to_thread.run_sync(
57              self.embeddings.embed_documents, [chunk.content for chunk in chunks]
58          )
59          if not vectors:
60              return []
61          await anyio.to_thread.run_sync(self._ensure_collection, len(vectors[0]))
62          records = [
63              {
64                  "id": str(chunk.id),
65                  "vector": vector,
66                  "knowledge_base_id": str(knowledge_base_id),
67                  "material_id": str(chunk.material_id),
68                  "content": chunk.content,
69                  "page_start": chunk.page_start,
70                  "page_end": chunk.page_end,
```

代码续

```
                "heading_path": chunk.heading_path,
            }
            for chunk, vector in zip(chunks, vectors, strict=True)
        ]
        await anyio.to_thread.run_sync(
            lambda: self.client.upsert(collection_name=self.collection_name, data=records)
        )
        return [str(chunk.id) for chunk in chunks]
```

chunks 是本次待索引的业务片段；knowledge_base_id 来自材料归属。

embed_documents 返回多条向量，zip(..., strict=True) 检查片段与向量数量对应。

返回的 list[str] 是各片段的向量主键，不是相似度分数。

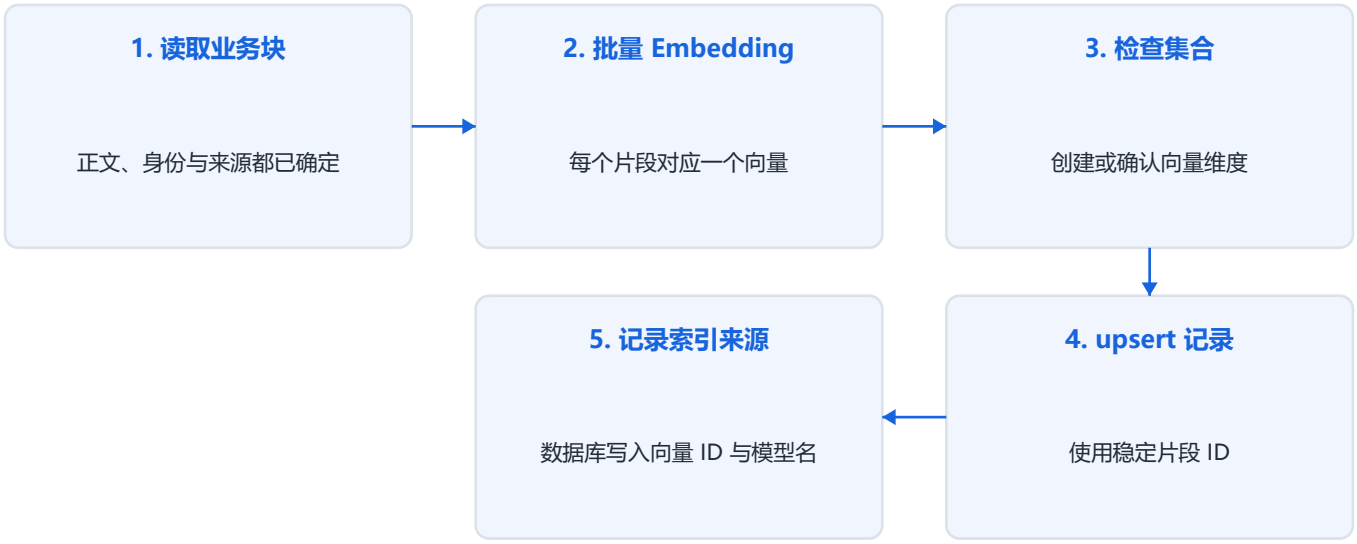


图 3-4 资料向量写入的实际数据关系。失败时，应定位在模型、Milvus 还是数据库阶段。

源码使用 `anyio.to_thread.run_sync` 把同步 Embedding 与 Milvus 调用放到工作线程。异步函数中的 await 允许事件循环在等待时处理其他事情，但不会把外部请求自动变快，也不会自动增加模型供应商配额。看到 async、await 和线程桥接时，先辨认它们解决的是阻塞调度问题。

方法	参数与返回	读代码时重点
upsert	knowledge_base_id + chunks → 向量 ID 列表	正文和元数据怎样与向量一一对应
_ensure_collection	dimensions → 无返回值	维度不匹配时失败，不悄悄覆盖集合
delete_material	material_id → 无返回值	按材料归属删除索引记录
delete_stale_chunks	material_id + keep_ids → 无返回值	只保留本次有效块，清理旧主键

表 3-4 索引方法的输入输出。下划线方法是内部辅助函数，通常由类的公开方法调用。

不要只验证向量数量

数量一致是基本契约，还应确认每条向量的维度、使用的模型和对应正文。真实检索质量需要问题与标准来源的标注集，不能通过 upsert 成功来推断。

3.6 search 与 top_k：范围先固定，再取相似候选

`MilvusDocumentIndex.search` 接收 knowledge_base_id、query、limit，还支持可选 material_id，返回 `list[SearchHit]`。查询会先把用户问题向量化，再构造知识库过滤条件。可选材料 ID 会进一步缩小范围。这里使用 UUID 类型的业务身份，不把用户任意字符串直接当作过滤表达式。

源码节选：查询向量与范围过滤

定位：apps/api/study_agent/ingestion/vector_index.py，第 88-108 行。

```
88          exists = await anyio.to_thread.run_sync(self.client.has_collection, self.collection_name)
89          if not exists:
90              return []
91          vector = await anyio.to_thread.run_sync(self.embeddings.embed_query, query)
92          expression = f'knowledge_base_id == "{knowledge_base_id}"'
93          if material_id:
94              expression += f' and material_id == "{material_id}"'
95          results = await anyio.to_thread.run_sync(
96              lambda: self.client.search(
97                  collection_name=self.collection_name,
98                  data=[vector],
99                  filter=expression,
100                 limit=limit,
101                 output_fields=[
102                     "material_id",
103                     "content",
104                     "page_start",
105                     "page_end",
106                     "heading_path",
107                 ],
108             )
```

self.client 是同步 MilvusClient，collection_name 是资料集合名。

不存在集合时返回空列表；embed_query 只生成一条查询向量。

data=[vector] 是一条查询组成的批次；filter 限定归属；limit 是最多召回数量。

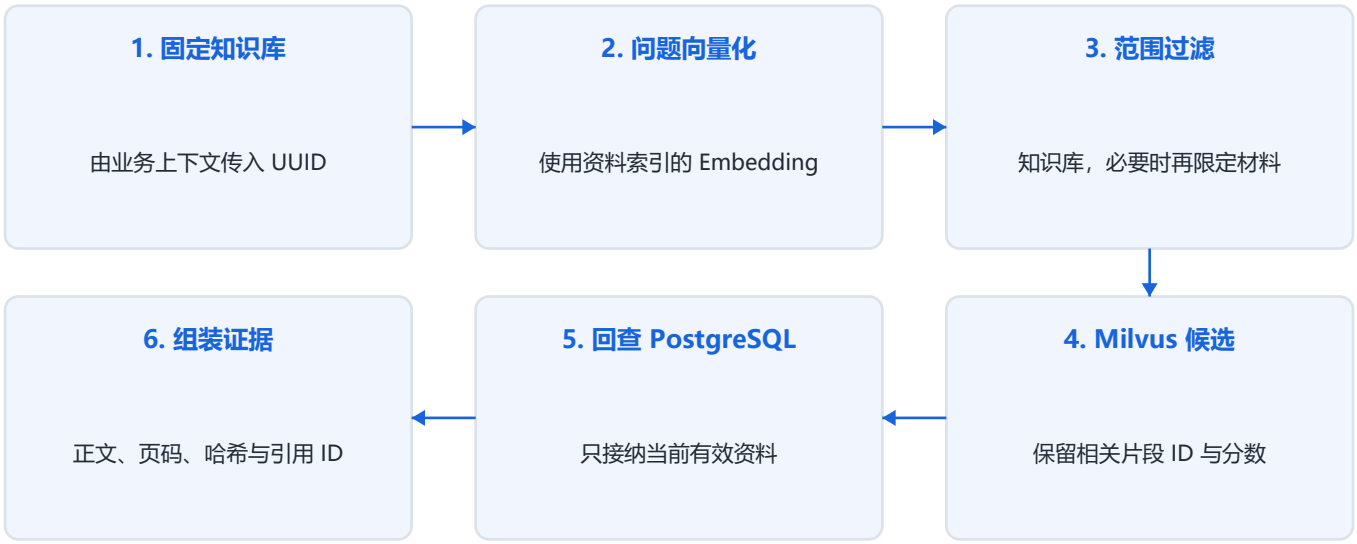


图 3-5 向量查询与业务证据的完整关系。Milvus 提供候选，PostgreSQL 提供当前业务事实。

辅导材料检索当前以知识库为范围，不会因用户问题里提到另一知识库名称就切换范围。工具的 top_k 默认 5，参数范围 1～5。索引的 limit 是候选数量，而后续数据库复核可能丢弃记录，所以最终证据数可以小于 top_k；代码没有在过滤后继续补召回到固定数量。

SearchHit 保留 chunk_id、material_id、候选正文、score 和来源字段。应用检索服务回查数据库后，会用数据库当前正文重新组装证据，并保持候选顺序。这意味着索引缓存的内容不是最终引用真相。删除或重新处理资料后，即使旧候选暂时仍出现，也应通过当前数据库记录挡住失效来源。

调大 k 的边界

更多候选可能补足遗漏，也可能增加噪声与上下文成本。当前没有 BM25、重排模型或最低分阈值；调整 k 不能自动替代这些机制。官方过滤说明：https://milvus.io/docs/filtered-search.md

3.7 从候选到证据：归属校验与失效复核为什么做两次？

`LearningMaterialRetrieval.search` 首先通过数据库关联检查：片段必须仍存在，材料必须属于当前知识库、parse_status 必须为 ready，候选中的 material_id 也要匹配当前材料。通过这些条件后，`evidence_from_rows` 把数据库记录转换为工具可以返回的证据字典。分数高只是候选相关，并不能绕过这些业务条件。

源码节选：生成有版本的引用证据

定位：apps/api/study_agent/application/retrieval.py，第 15-29 行。

```python
def evidence_from_rows(
    chunk: DocumentChunkModel, material: MaterialModel, quote: str | None = None
) -> dict[str, Any]:
    excerpt = (quote if quote and quote in chunk.content else chunk.content)[:1200]
    return {
        "evidence_id": f"chunk:{chunk.id}:{chunk.content_hash}",
        "material_id": str(material.id),
        "chunk_id": str(chunk.id),
        "title": material.title,
        "page_start": chunk.page_start,
        "page_end": chunk.page_end,
        "quote": excerpt,
        "content_hash": chunk.content_hash,
        "valid": True,
    }
```

chunk 与 material 是数据库查询得到的一组记录，不是模型自行编造的对象。

quote 只有确实存在于 chunk.content 时才采用，否则回退到当前正文；截取前 1200 字符。

evidence_id 包含片段 ID 和正文哈希，让引用指向具体内容版本。

证据字段 / 检查	含义	防止什么问题
material_id + chunk_id	资料与片段身份	把其他材料候选混入当前回答
title + page_start/page_end	展示与定位来源	只有答案，没有可查看出处
quote	最多 1200 字符的当前原文摘录	工具结果无限增大或引用不存在的句子
content_hash	正文版本	处理后内容变化，旧证据仍被当成当前证据
revalidate	再次查归属、ready、哈希与原文包含关系	检索后、提交前材料发生变化

表 3-5 证据要回答：来自哪里、引用了什么、是否仍然有效。valid=True 是组装时标记，提交前还需复核。

源码节选：提交前再次核对当前证据

定位：apps/api/study_agent/application/retrieval.py，第 65-83 行。

```python
    async def revalidate(self, knowledge_base_id: UUID, evidence: dict[str, Any]) -> bool:
        async with self.sessions() as session:
            row = (
                await session.execute(
                    select(DocumentChunkModel, MaterialModel)
                    .join(MaterialModel, DocumentChunkModel.material_id == MaterialModel.id)
                    .where(
                        DocumentChunkModel.id == UUID(evidence["chunk_id"]),
                        MaterialModel.id == UUID(evidence["material_id"]),
                        MaterialModel.knowledge_base_id == knowledge_base_id,
                        MaterialModel.parse_status == "ready",
                    )
                )
            ).first()
            return bool(
                row
                and row[0].content_hash == evidence["content_hash"]
                and evidence["quote"] in row[0].content
            )
```

knowledge_base_id 是当前业务范围，evidence 是已登记证据字典。

SQL 条件检查 chunk、material、知识库与 ready；随后比较哈希和 quote 原文包含关系。

返回 bool 表示当前证据是否有效；它不判断模型每一句解释是否符合事实。

检索后用户可能删除资料，或者资料被重新处理，正文版本发生变化。提交前的 `revalidate` 就是为了处理这种时间差。它能验证归属与引用版本，却不能证明回答的完整性或推理正确性。例如模型引用了正确握手原文，仍可能对原因做错误解释，这需要单独的答案质量检查。



图 3-6 真实题目详情页面。观察题干、解析与材料来源如何并列展示；界面可帮助你回看来源，但代码中的证据复核仍由服务端完成。

当前真实项目页面 / 现有业务数据 / 2026-10-07。题目、标准答案、解析与原文引文的真实审核界面

3.8 完整离线实验：检索过滤与证据组装分别验证

在项目根目录新建自己的 `ch03_dense.py`，复制下面代码，用 `apps/api/.venv/Scripts/python.exe` 运行。实验使用项目的 DeterministicEmbeddings 和 LangChain 内存向量库。A、B 两个知识库都有完全相同的 TCP 文本，故意设置这种输入，是为了检验“相似”与“允许检索的范围”是否分开处理。

完整可运行实验：内存检索与 metadata 过滤

```
from langchain_core.documents import Document
from langchain_core.vectorstores import InMemoryVectorStore

from study_agent.ingestion.embeddings import DeterministicEmbeddings

embedding = DeterministicEmbeddings(dimensions=32)
store = InMemoryVectorStore(embedding=embedding)
documents = [
    Document(id="tcp-a", page_content="TCP 用三次握手建立连接。",
             metadata={"knowledge_base_id": "A"}),
```

代码续

```
    Document(id="redis-a", page_content="Redis 保存学习会话快照。",
             metadata={"knowledge_base_id": "A"}),
    Document(id="tcp-b", page_content="TCP 用三次握手建立连接。",
             metadata={"knowledge_base_id": "B"}),
]
ids = store.add_documents(documents)
query = "TCP 用三次握手建立连接。"
vector = embedding.embed_query(query)
assert vector == embedding.embed_query(query)
hits = store.similarity_search_with_score(
    query, k=3,
    filter=lambda doc: doc.metadata["knowledge_base_id"] == "A",
)
assert len(vector) == 32
assert hits[0][0].id == "tcp-a"
assert all(doc.metadata["knowledge_base_id"] == "A" for doc, _ in hits)
print("dimension:", len(vector))
print("stored_ids:", ids)
print("filtered_ids:", [doc.id for doc, _ in hits])
print("scores:", [round(score, 4) for _, score in hits])
print("external_requests:", 0)
```

Document.id 固定文档身份，metadata 保存教学知识库标识。

add_documents 在内存中调用本地向量化；filter 回调只保留 A。

similarity_search_with_score 返回 (Document, score)；断言检查结果归属和固定输入排序。

结果解释：本次输出 dimension: 32；stored_ids: ['tcp-a','redis-a','tcp-b']；filtered_ids: ['tcp-a','redis-a']；scores: [1.0,0.0]；external_requests: 0。

相同查询与正文得到 1.0，证明本地确定性表示和相似度路径能工作；B 中同样高相似的文本被过滤掉，证明教学范围条件生效。Redis 文本仍可能作为次近候选出现，说明 k=3 不等于三个结果都能回答问题。这些固定数值不是生产模型的语义检索指标，也没有验证真实 Milvus 服务。

再新建自己的 `ch03_evidence.py`，运行下面证据实验。它创建两个不连接数据库的 SQLAlchemy 模型对象，直接调用生产 `evidence_from_rows`。这样可以单独观察 quote 回退、页码与版本 ID，而不需要先搭建完整辅导服务。真正的数据库归属与 revalidate 查询仍需集成验收。

完整可运行实验：生产证据函数的输入输出

```
from hashlib import sha256
from uuid import UUID

from study_agent.application.retrieval import evidence_from_rows
from study_agent.infrastructure.models import DocumentChunkModel, MaterialModel

text = "TCP 用三次握手建立连接，确认双方收发能力。"
chunk = DocumentChunkModel(
    id=UUID(int=1), material_id=UUID(int=2), chunk_index=0,
    content=text, content_hash=sha256(text.encode("utf-8")).hexdigest(),
    token_count=24, page_start=3, page_end=3, heading_path=["网络"],
```

代码续

```
)
material = MaterialModel(
    id=UUID(int=2), knowledge_base_id=UUID(int=3), title="网络讲义",
    parse_status="ready",
)
good = evidence_from_rows(chunk, material, quote="三次握手")
fallback = evidence_from_rows(chunk, material, quote="材料中不存在的句子")
assert good["quote"] == "三次握手"
assert fallback["quote"] == text
assert good["content_hash"] == chunk.content_hash
print("quote:", good["quote"])
print("page:", good["page_start"])
print("fallback_to_content:", fallback["quote"] == text)
print("version_in_id:", good["evidence_id"].endswith(chunk.content_hash))
print("database_requests:", 0)
```

模型对象仅在内存中构造；没有 Session、commit 或数据库网络请求。

第一种 quote 确实在原文中；第二种不存在，因此回退到原文。

evidence_id 末尾包含 content_hash，page_start 保留为 3。

结果解释：本次输出 quote: 三次握手；page: 3；fallback_to_content: True；version_in_id: True；database_requests: 0。

实验层次

内存检索实验验证转换、排序与过滤的基本机制；证据实验验证一个生产函数的输出。二者都不证明数据库权限、供应商模型质量或 Web 全链路效果。

3.9 双索引与后续优化：先准确描述已有能力

项目配置了 study_documents 和 study_questions 两个集合。前者索引资料片段，服务学习材料检索；后者索引题干、答案与解析等题目文本，支持题目相关的数据准备。它们按业务对象划分。两套集合并不等于同时做 Dense 和 BM25，也不代表当前查询执行了融合排序。



图 3-7 真实项目题库页面。题目是独立业务对象，和资料片段有不同字段；资料索引与题目索引按对象区分。

当前真实项目页面 / 现有业务数据 / 2026-10-07。题库审核状态及原文引用数量

飞书课程先用小实验解释向量化，再展示 Milvus 字段、索引、过滤和混合召回。本项目可沿用这种讲解顺序，同时把实现范围写准确：当前资料检索只有 FLOAT_VECTOR、AUTOINDEX、COSINE 和知识库过滤，没有稀疏字段、BM25 函数、RRF 融合或 Cross-Encoder 重排。

后续优化应从失败问题出发：若专有名词经常漏召回，可考虑关键词或混合检索；若召回包含相关与无关片段，可评估重排与阈值；若答案需要跨块上下文，可考虑邻接块扩展。先建立问题、标准来源与期望答案，再比较变更前后，才能判断优化是否有效。没有标注和对照实验时，不写提升百分比。

- 去掉实验 filter，确认 B 的相同文本是否进入结果，并解释风险。
- 把查询改成资料中没有的主题，观察为什么仍可能返回候选。
- 改维度为 64，观察本地分数变化，说明分数不能跨表示直接比较。
- 给证据函数传 1500 字符正文，验证 quote 截断；再思考关键句位于后半段时会怎样。

本章复盘应包含一次可复现查询、实际过滤结果和证据字段解释。你可以画出“索引候选 → PostgreSQL 当前记录 → 引用证据”的关系，说明为什么归属检查与语义相关性是不同责任。掌握这个关系后，再学习 Agent 的工具循环，就不会把模型的自由选择误认为可以自由跨越知识库范围。

3.10 综合练习与参考思路

练习 1：换 Embedding 模型，维度相同就可以复用旧索引吗？

参考思路：不能直接假定兼容；不同模型的向量空间可能不同，应保持写入和查询一致，并评估全量重新向量化。

练习 2：top_k=5 为什么最终只有 3 条证据？

参考思路：Milvus 候选可能不足，或者 PostgreSQL 归属、ready、材料匹配复核丢弃部分候选；当前没有补召回到固定数量。

练习 3：COSINE 得分 0.8 是否表示答案有 80% 正确率？

参考思路：不是；它是这次向量比较的相似度，答案质量和推理正确性需要其他验证。

练习 4：资料和题目两个索引，为什么不算混合检索？

参考思路：它们按业务对象划分；混合检索需要多个召回信号及融合逻辑，当前资料查询没有 Dense/BM25 双路召回。

练习 5：revalidate 通过能证明回答每一句都正确吗？

参考思路：不能；它证明当前来源归属、状态、正文版本与引用关系有效，不评判所有语义解释。

3.11 源码阅读清单与下一步

文件或入口	关注的函数 / 类	阅读时回答的问题
apps/api/study_agent/ingestion/embeddings.py	Embeddings 实现 / build_embeddings	查询和批量文档怎么向量化，哪些实现会调用外部服务？
apps/api/study_agent/ingestion/vector_index.py	MilvusDocumentIndex	Schema、维度、COSINE、过滤和 SearchHit 怎样对应？
apps/api/study_agent/application/retrieval.py	LearningMaterialRetrieval / evidence_from_rows	候选如何成为当前有效证据，为什么要再次复核？
apps/api/study_agent/agents/tutor/tools.py	SearchInput / search_learning_materials	top_k 与知识库边界从哪里传入工具？
apps/api/study_agent/question_generation/vector_index.py	MilvusQuestionIndex	题目索引与资料索引是业务对象区别还是召回算法区别？
apps/api/study_agent/config.py	Settings	向量提供者、维度与集合名称有哪些默认值？

下一章：第 4 章转入固定题目生成链：材料怎样组织为提示词，模型输出怎样成为可校验的结构化题目。辅导 Agent 的动态检索工具在第 5 章继续展开。

主要参考：本地飞书 LangChain、Agent、RAG 与 LangGraph 课程的教学组织方式，以及当前项目源码。具体框架接口以锁定环境与官方文档为准。